

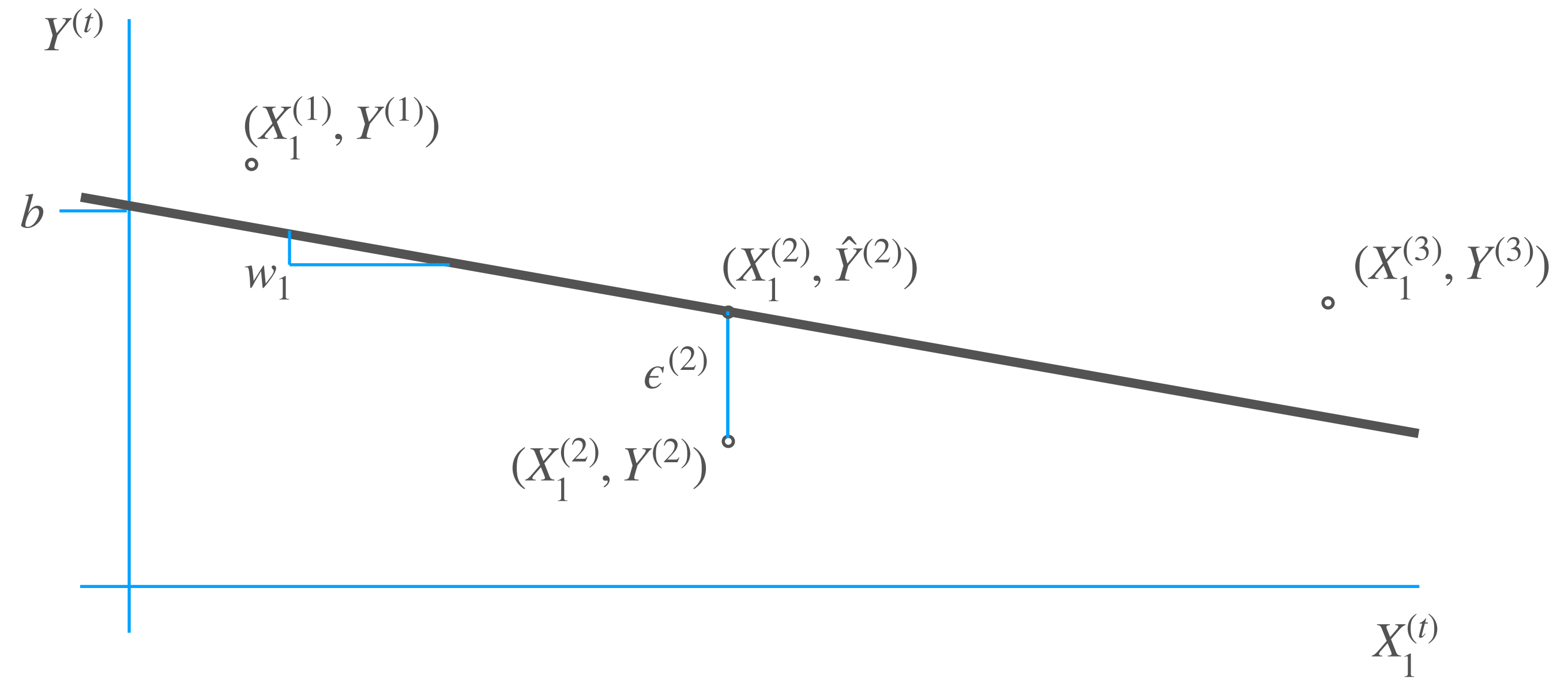
Linear regression



10-701 Introduction to Machine Learning
Geoff Gordon

Linear regression

w = slope (in each feature),
 b = intercept

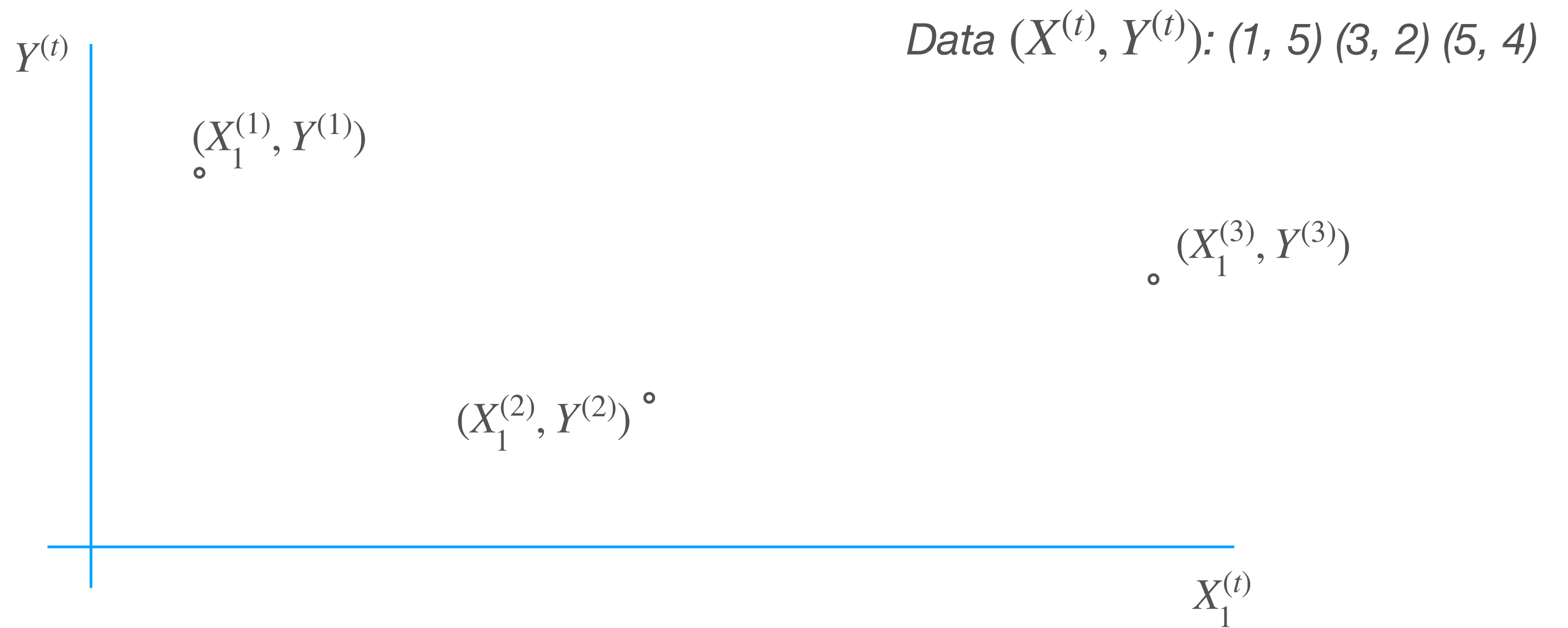


- Regression = supervised prediction of *real* labels
 - ▶ data: features $X^{(t)} \in \mathbb{R}^d$, labels $Y^{(t)} \in \mathbb{R}$, $t = 1 : N$
- *Linear* regression = predict a linear function of features
 - ▶ model: predict $\hat{Y}^{(t)} = w \cdot X^{(t)} = \sum_i w_i X_i^{(t)}$ or $w \cdot X^{(t)} + b$
 - ▶ can omit b if we have a constant feature

Linear regression

- Objective:
 - ▶ minimize sum of squared errors:
$$\frac{1}{2} \sum_{t=1}^N \underbrace{(Y^{(t)} - \hat{Y}^{(t)})^2}_{\epsilon^{(t)}}$$
 - ▶ plus optional regularizer (see below)
- E.g., minimize
$$L(w) = \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - \hat{Y}^{(t)})^2 + \frac{\lambda}{2} \|w\|_2^2$$

Tiny example

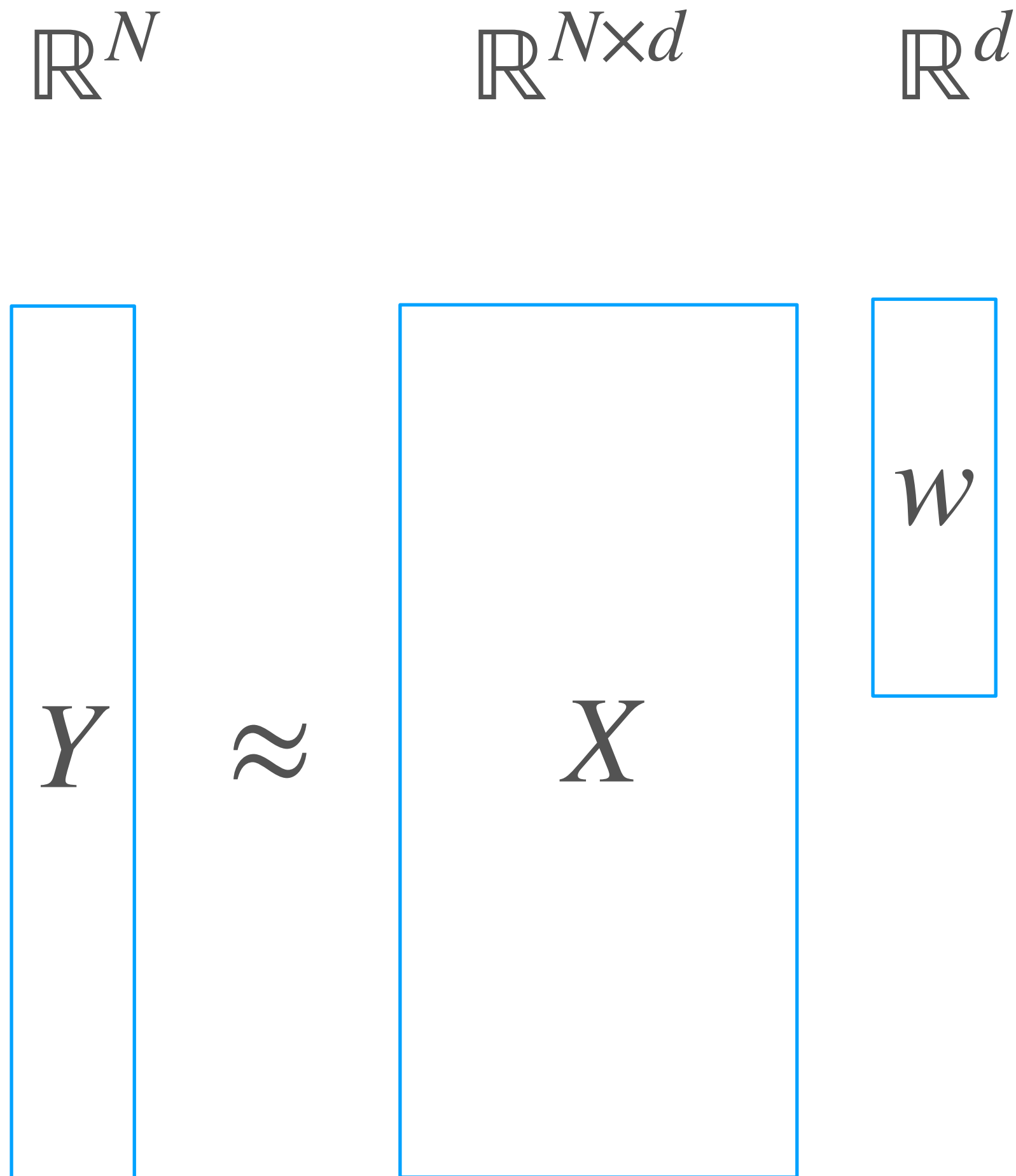


$$L(w, b) =$$

$$\nabla_b L(w, b) =$$

$$\nabla_w L(w, b) =$$

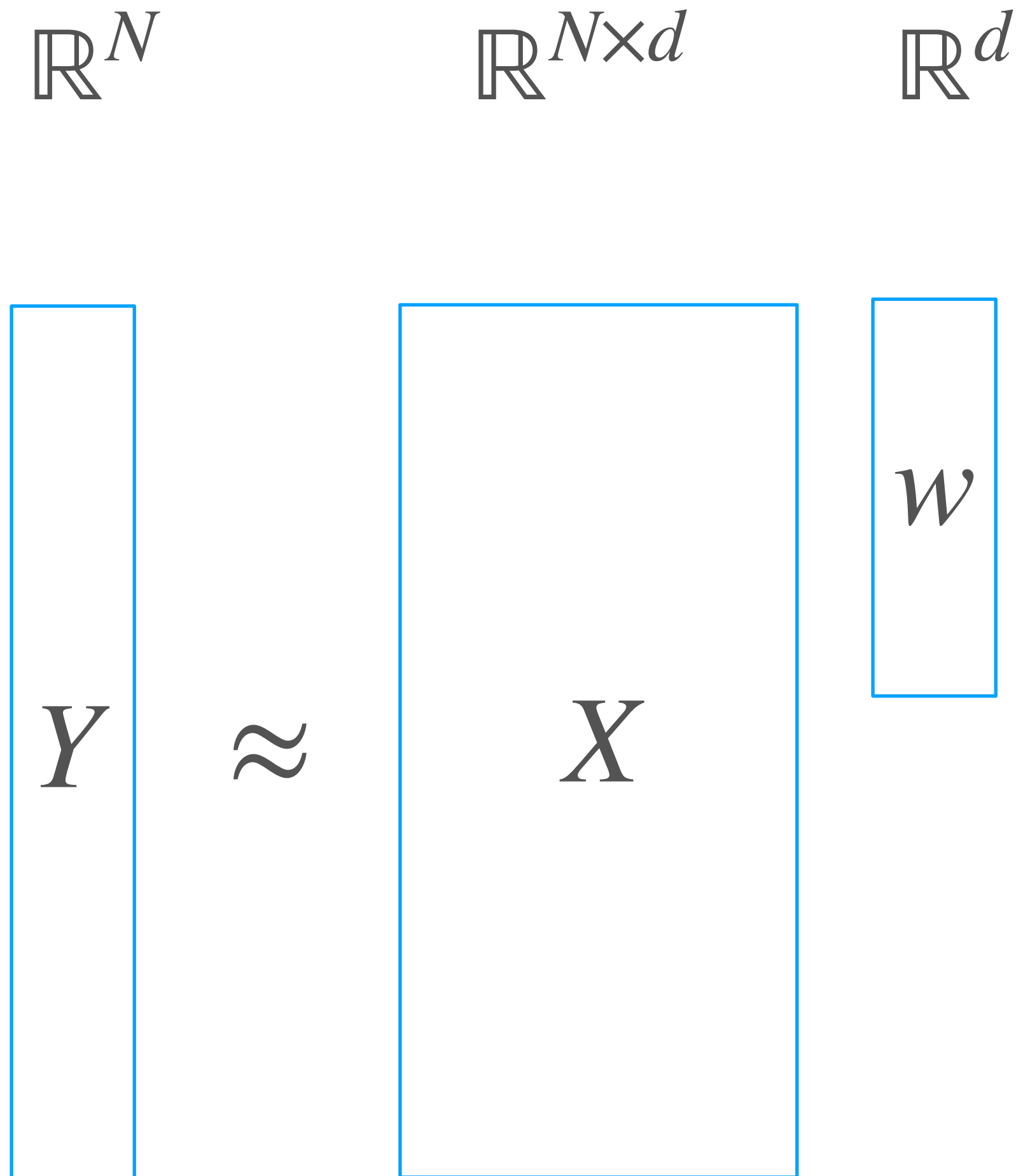
Schematic



features X			labels y
Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
Yes	Low	Normal	0.1
No	Medium	Normal	0.2
No	Low	Abnormal	10.3
Yes	Medium	Normal	8.7
Yes	High	Abnormal	13.1
No	Low	Normal	0.1
No	Medium	Normal	0.4
Yes	Medium	Abnormal	9.6

- data matrix X ,
label vector Y ;
rows are $X^{(t)}$, $Y^{(t)}$

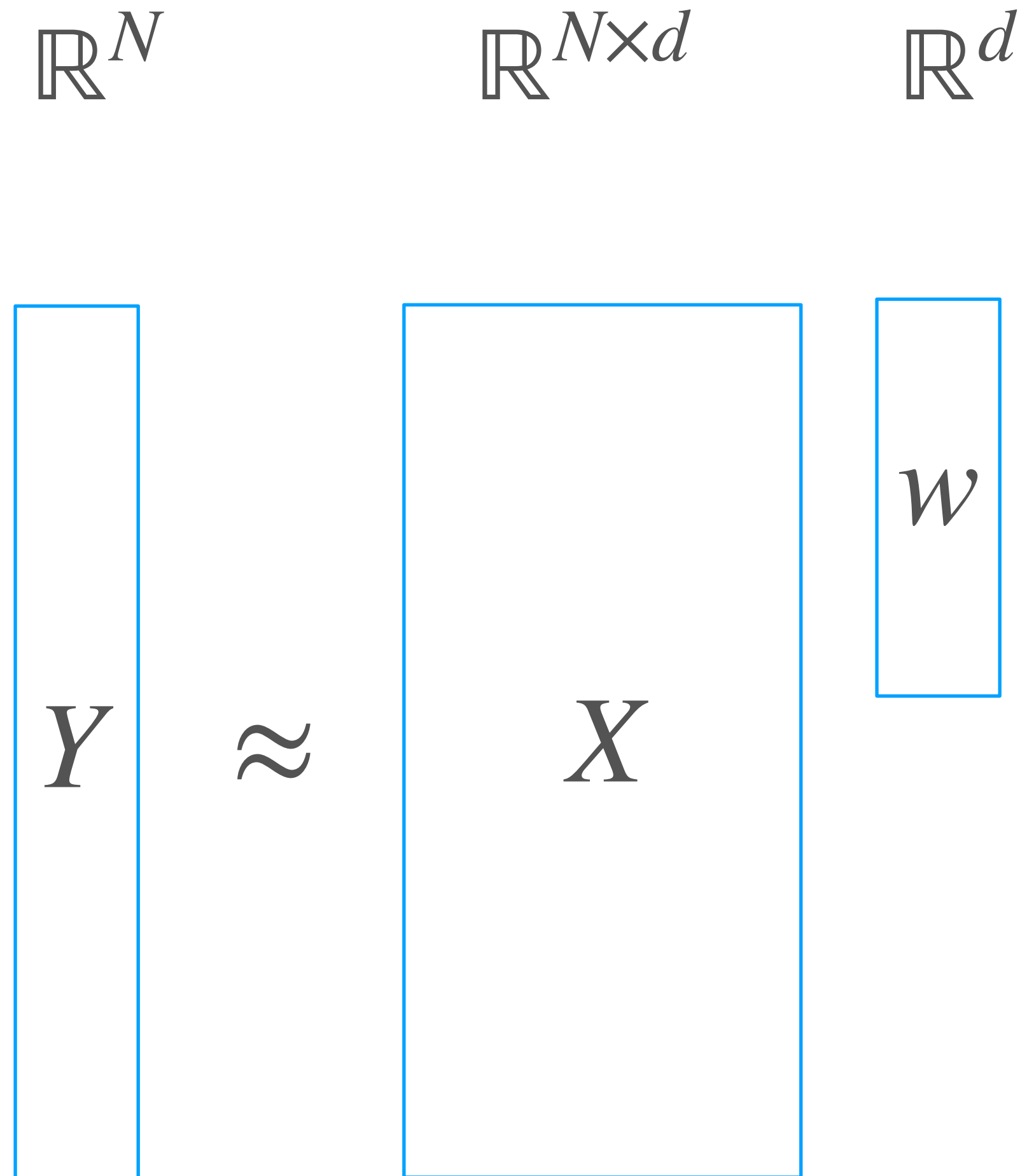
Schematic



	features X			labels y
	Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
I	Yes	Low	Normal	0.1
I	No	Medium	Normal	0.2
I	No	Low	Abnormal	10.3
I	Yes	Medium	Normal	8.7
I	Yes	High	Abnormal	13.1
I	No	Low	Normal	0.1
I	No	Medium	Normal	0.4
I	Yes	Medium	Abnormal	9.6

- data matrix X ,
label vector Y ;
rows are $X^{(t)}$, $Y^{(t)}$

Schematic

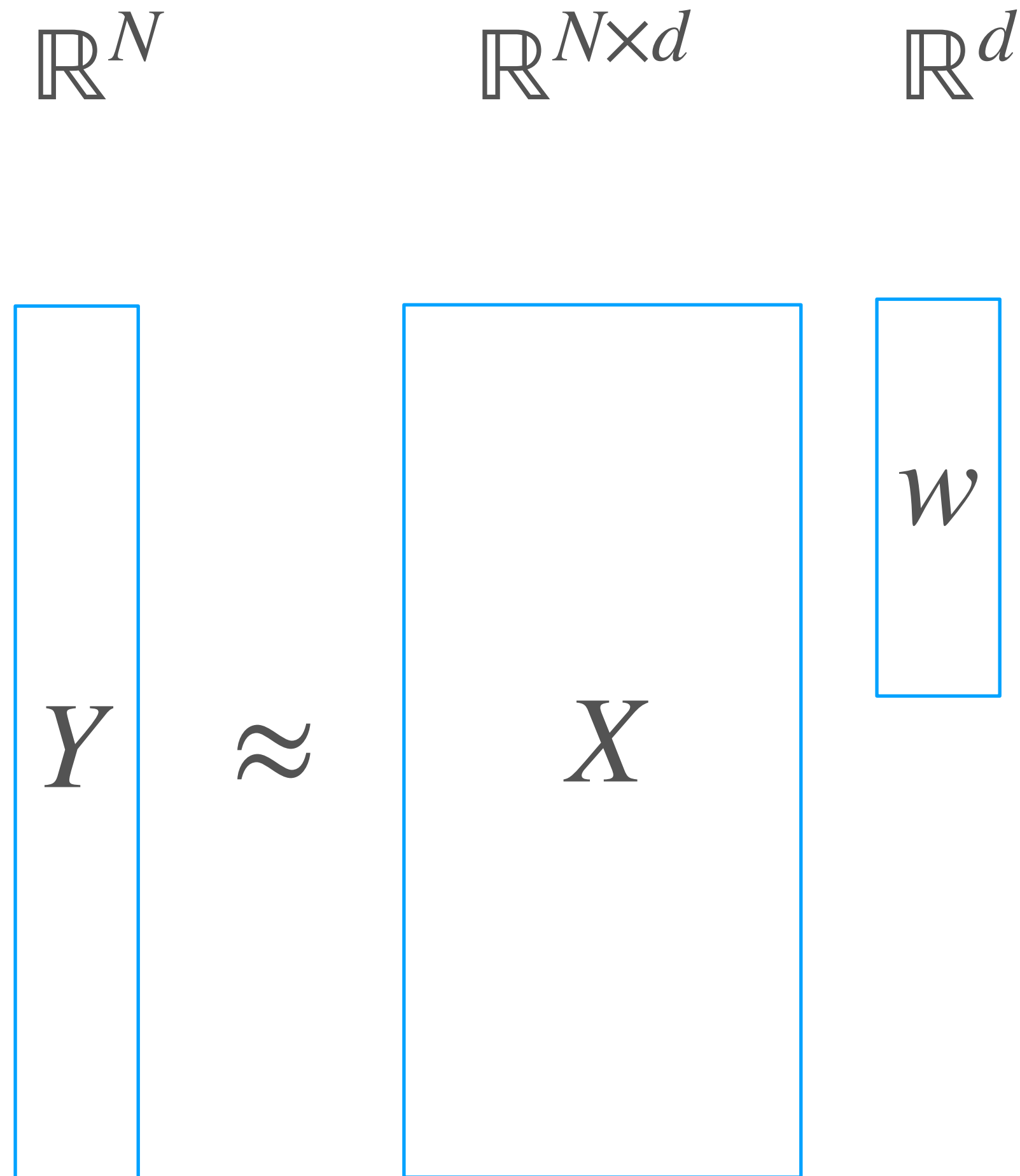


$$L(w) = \|Y - Xw\|_F^2 \quad [+ R(w)]$$

	features X			labels y
	Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
I	Yes	Low	Normal	0.1
I	No	Medium	Normal	0.2
I	No	Low	Abnormal	10.3
I	Yes	Medium	Normal	8.7
I	Yes	High	Abnormal	13.1
I	No	Low	Normal	0.1
I	No	Medium	Normal	0.4
I	Yes	Medium	Abnormal	9.6

- data matrix X , label vector Y ; rows are $X^{(t)}$, $Y^{(t)}$

Schematic



features X			labels y
Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
			0.1
			0.2
			10.3
			8.7
			13.1
			0.1
			0.4
			9.6

Wait, these need to be numbers

$$L(w) = \|Y - Xw\|_F^2 \quad [+ R(w)]$$

- data matrix X , label vector Y ; rows are $X^{(t)}, Y^{(t)}$

Turn categorical features into numeric

Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
Yes	Low	Normal	0.1
No	Medium	Normal	0.2
No	Low	Abnormal	10.3
Yes	Medium	Normal	8.7
Yes	High	Abnormal	13.1
No	Low	Normal	0.1
No	Medium	Normal	0.4
Yes	Medium	Abnormal	9.6

- Categorical = list of discrete possible values
 - ▶ e.g., weather = {rainy, sunny, windy}

***Turn
categorical
features into
numeric***

Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
1	Low	Normal	0.1
0	Medium	Normal	0.2
0	Low	Abnormal	10.3
1	Medium	Normal	8.7
1	High	Abnormal	13.1
0	Low	Normal	0.1
0	Medium	Normal	0.4
1	Medium	Abnormal	9.6

- Categorical = list of discrete possible values
 - ▶ e.g., weather = {rainy, sunny, windy}

*Turn
categorical
features into
numeric*

Family History	Resting Blood Pressure	Cholesterol			Cost (k\$)
1	Low		0	Low	0.1
0	Medium		0	Low	0.2
0	Low	A	1	Normal	10.3
1	Medium		0	Low	8.7
1	High	A	1	Normal	13.1
0	Low		0	Low	0.1
0	Medium		0	Low	0.4
1	Medium	A	1	Normal	9.6

- Categorical = list of discrete possible values
 - ▶ e.g., weather = {rainy, sunny, windy}

*Turn
categorical
features into
numeric*

Family History	Resting Blood Pressure	Cholesterol	Cost (k\$)
1	0 0	0	0.1
0	1 0	0	0.2
0	0 0	A 1	10.3
1	1 0	0	8.7
1	0 1	A 1	13.1
0	0 0	0	0.1
0	1 0	0	0.4
1	1 0	A 1	9.6

- Categorical = list of discrete possible values
 - ▶ e.g., weather = {rainy, sunny, windy}

One-hot encoding

*metaphor: like
making one wire
“hot”, i.e.,
having an
applied voltage*

- Tool: *one-hot* encoding
 - ▶ create one numerical feature for each possible value
 - ▶ optionally, skip feature for default value
 - ▶ default $\rightarrow (0,0,\dots,0)$
 - ▶ nondefault $\rightarrow$ set one feature to 1, rest to 0
- E.g.:
 - ▶ no $\rightarrow (0)$ yes $\rightarrow (1)$
 - ▶ rainy $\rightarrow (0,0)$ sunny $\rightarrow (1,0)$ windy $\rightarrow (0,1)$
 - ▶ rainy $\rightarrow (1,0,0)$ sunny $\rightarrow (0,1,0)$ windy $\rightarrow (0,0,1)$

MOO



- Linear regression
 - ▶ model = linear functions of features (+ constant)
 - ▶ objective = sum of squared errors (+ regularizer)
 - ▶ optimizer = exact (differentiate and set to zero, solve with Gaussian elimination)
 - ▶ or several other options

Regularizers

- Common choices:
 - ▶ $\|w\|_2^2$ called *ridge regression*
 - ▶ $\|w\|_1$ called *LASSO*
 - ▶ if we use both, *elastic net*

- E.g., minimize

$$L(w) = \frac{1}{2} \sum_{t=1}^T (Y^{(t)} - \hat{Y}^{(t)})^2 + \frac{\lambda}{2} \|w\|_2^2 + \beta \|w\|_1$$

Bias- variance tradeoff

- Why regularize?
- Trade off underfitting vs. overfitting
 - ▶ underfitting is often called ***bias***: the correct answer is not an option, so we pick something that might be far from it
 - ▶ overfitting shows up as ***variance*** of parameters w : more choices → more variability in which one we select

Overfitting, bias, variance

```
n = 10
k = 7
X = np.random.normal(size=(n, k))
Y = np.random.normal(size=(n, 1))
w, _, _, _ = np.linalg.lstsq(X, Y, rcond=None)
(np.linalg.norm(w), np.mean((Y - X@w)**2))
```

- What is the smallest MSE this code should return if there were no selection bias?
- What is the actual best w ?

Overfitting, bias, variance

```
n = 10
k = 7
X = np.random.normal(size=(n, k))
Y = np.random.normal(size=(n, 1))
w, _, _, _ = np.linalg.lstsq(X, Y, rcond=None)
(np.linalg.norm(w), np.mean((Y - X@w)**2))
```

- What is the smallest MSE this code should return if there were no selection bias?
- What is the actual best w ?

```
# three runs (n=10, k=7):
```

```
(1.729258520013365, 0.09403950328236421)
(1.9706932994777069, 0.07379394277037196)
(3.591128626783351, 0.47238275606461944)
```

Overfitting, bias, variance

```
n = 10
k = 7
X = np.random.normal(size=(n, k))
Y = np.random.normal(size=(n, 1))
w, _, _, _ = np.linalg.lstsq(X, Y, rcond=None)
(np.linalg.norm(w), np.mean((Y - X@w)**2))
```

- What is the smallest MSE this code should return if there were no selection bias?
- What is the actual best w ?

```
# three runs (n=10, k=7):
```

```
(1.729258520013365, 0.09403950328236421)
(1.9706932994777069, 0.07379394277037196)
(3.591128626783351, 0.47238275606461944)
```

```
# three runs (n=20, k=2):
```

```
(0.20006773436081007, 0.8932440732695813)
(0.3765035073938464, 0.9200354120567253)
(0.5764201246357761, 0.7614382010070438)
```


Effect of regularization

- Regularization attempts to limit overfitting by preferring simpler models (smaller weights)

three runs (n=10, k=7, lambda=0):

(1.729258520013365, 0.09403950328236421)
(1.9706932994777069, 0.07379394277037196)
(3.591128626783351, 0.47238275606461944)

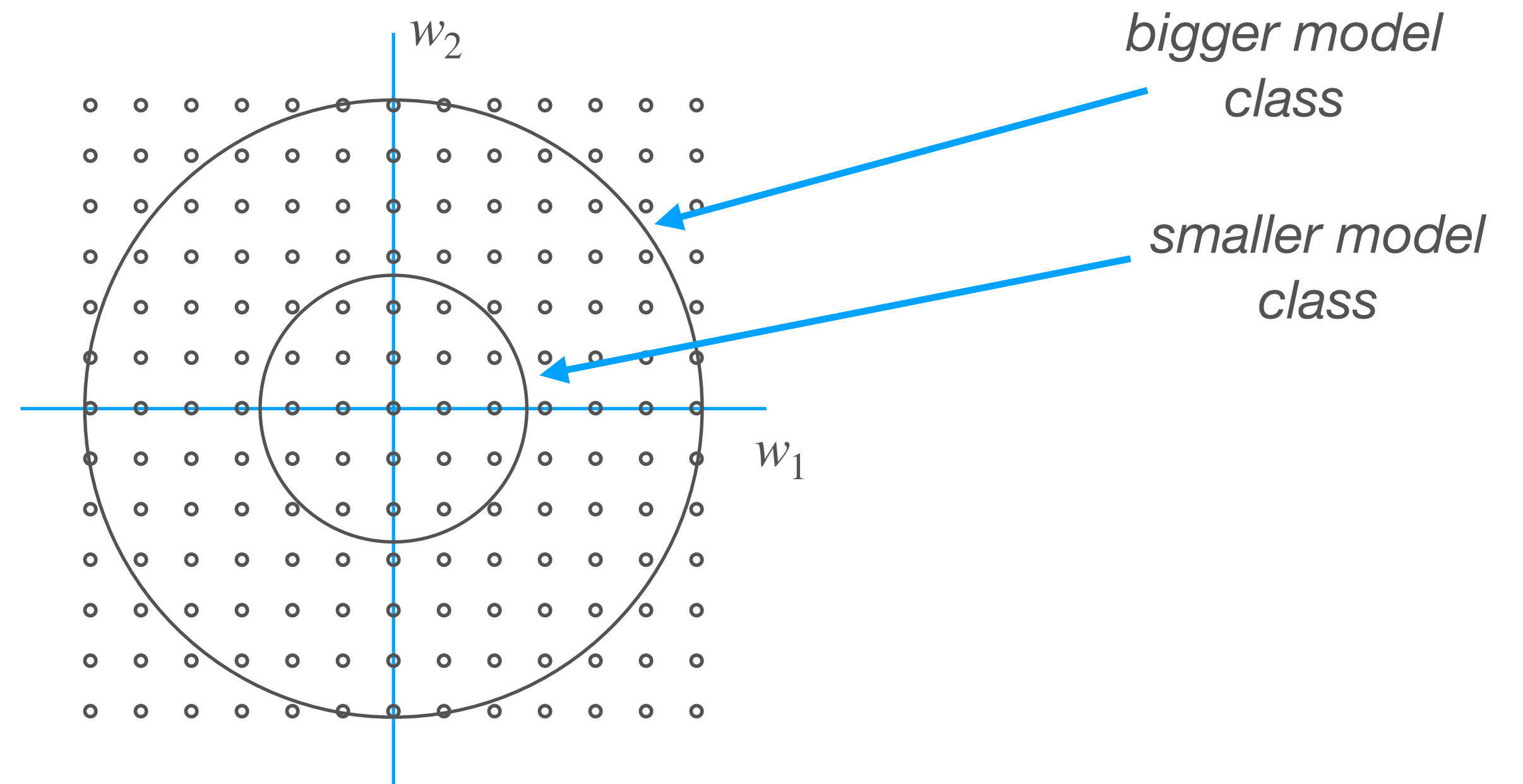
three runs (n=10, k=7, lambda=1):

(0.5961635084865388, 0.20898544573161687)
(0.6399685598204747, 0.30440651674916513)
(0.9915710819788339, 0.19630426412988047)

three runs (n=10, k=7, lambda=10):

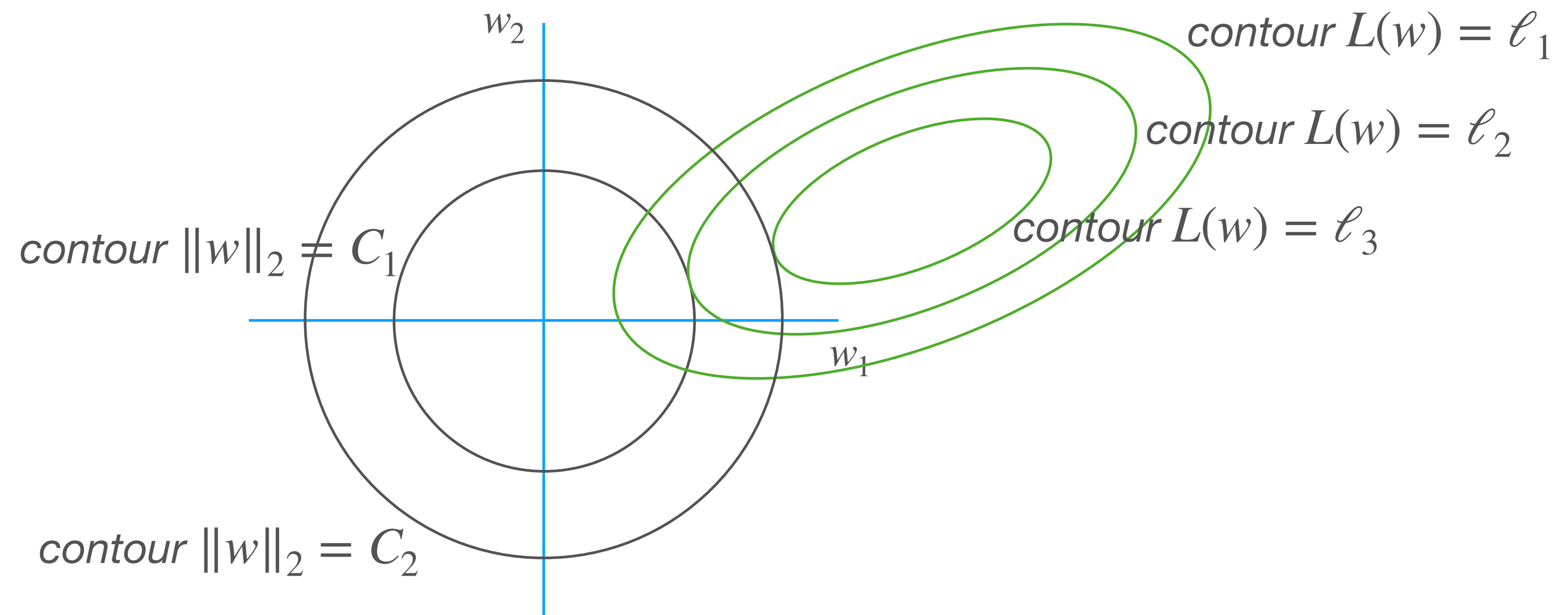
(0.18652470206125477, 0.20138831819271244)
(0.3682010478765221, 0.4279430122544764)
(0.46978624375583933, 0.44861948640322524)

Model complexity



- Measure complexity of our model class as the number of *effectively distinct* weight vectors
 - ▶ if two weight vectors are very close together, they make almost the same predictions
 - ▶ count only vectors in a grid w/ separation ϵ
- More w s if we have *more features* or *bigger weights*
 - ▶ regularization can limit both

Model selection



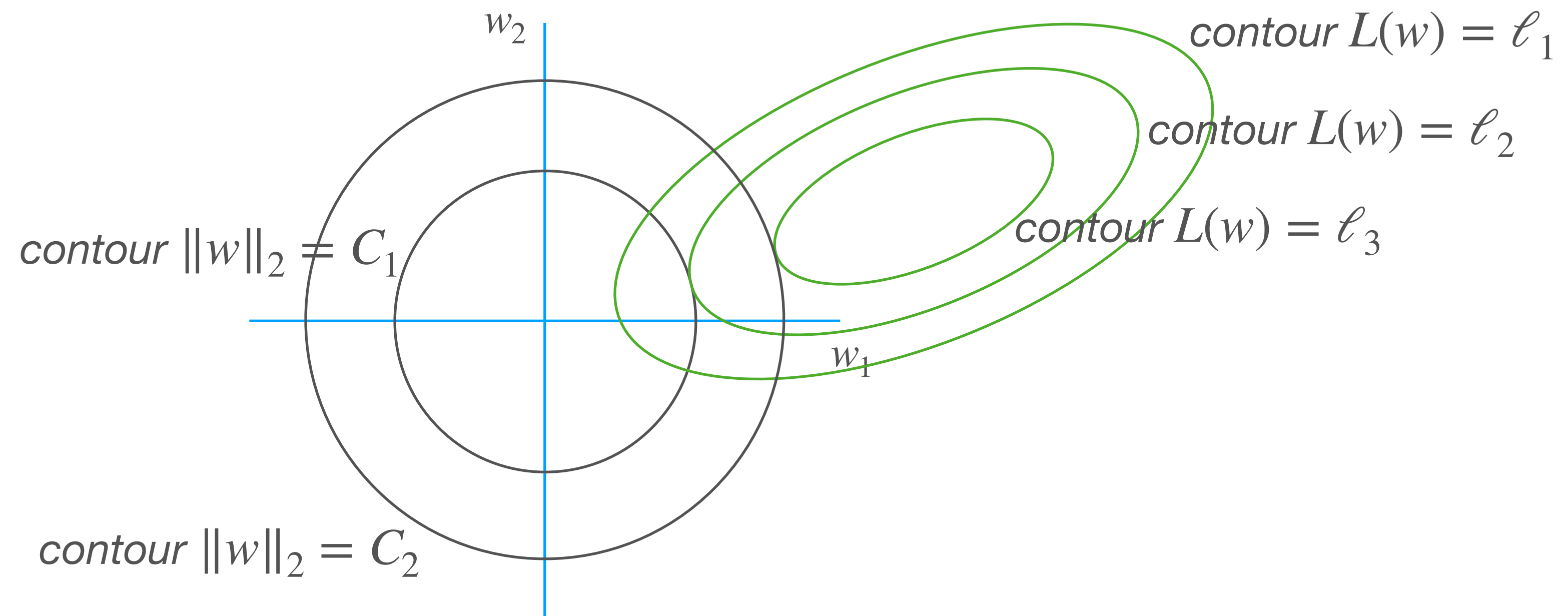
- Limit Euclidean norm of weight vector:

$$\min_w L(w) = \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - X^{(t)} \cdot w)^2 \quad \text{s.t.} \quad \|w\|_2 \leq C$$

- Model selection = picking C

Penalty vs. constraint

Idea: it's easier to pick a good λ ahead of time



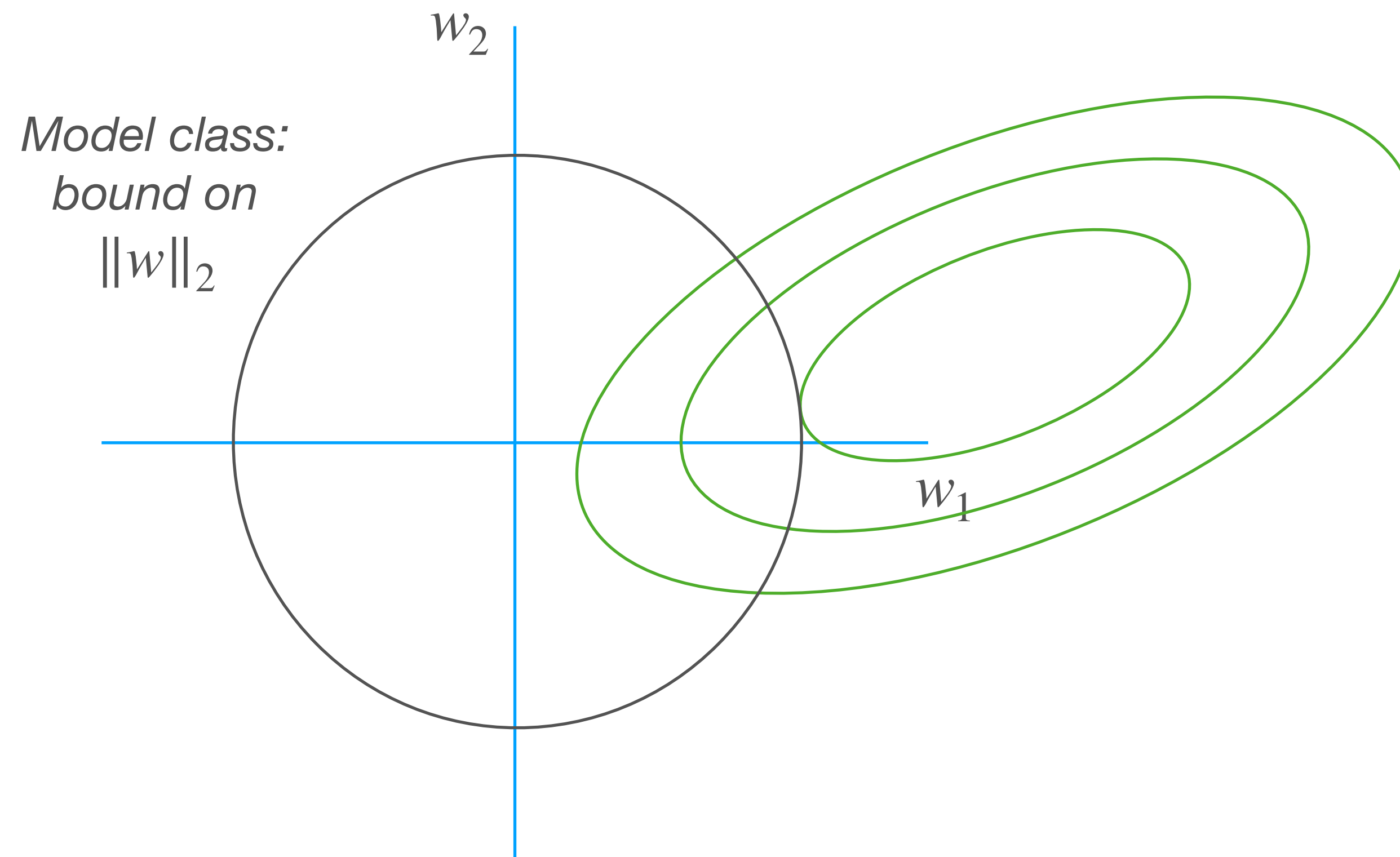
- Instead of constrained minimization

$$\min_w L(w) = \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - X^{(t)} \cdot w)^2 \quad \text{s.t.} \quad \|w\|_2 \leq C$$

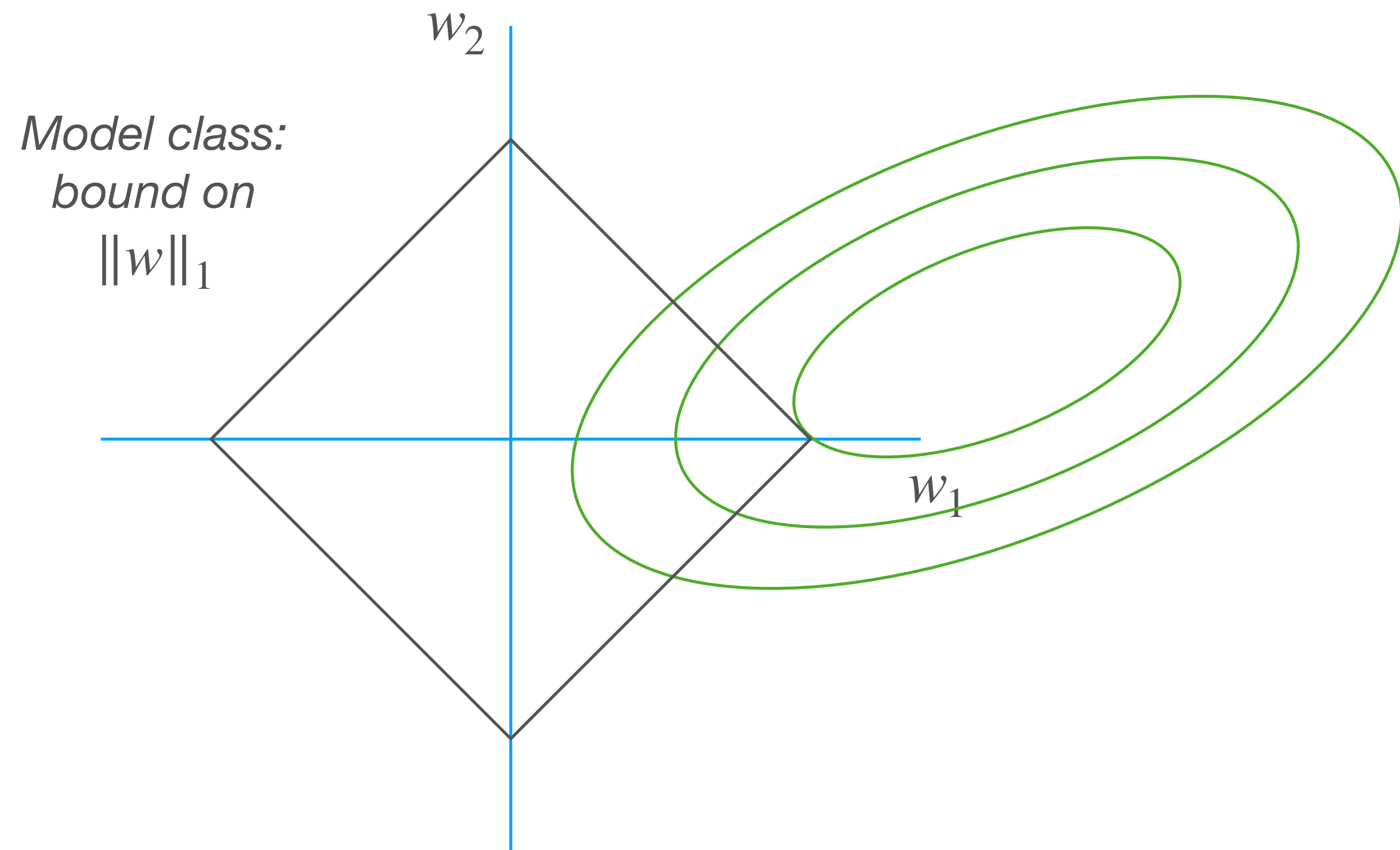
- we can try to directly trade off $L(w)$ and $\|w\|_2$ in optimizer (avoid outer model selection loop):

$$\min_w L(w) = \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - X^{(t)} \cdot w)^2 + \frac{\lambda}{2} \|w\|_2^2$$

***L_1 norm and
count of
nonzeros
are related***



***L_1 norm and
count of
nonzeros
are related***



Pre- processing

- Linear models are extremely useful, but can be sensitive to how the features are expressed
 - ▶ also true of nonlinear models
- Extremely common to modify features before training — called *pre-processing*
 - ▶ e.g., one-hot encoding of categorical features
- Large toolbox of possible ways to pre-process
 - ▶ each tool is designed to fix a potential problem
 - ▶ use any or all of them, as the situation calls for
 - ▶ but don't use them unless they're needed
- Bundle together into a *feature transform*

Ordered features

- Sometimes discrete features have a built-in order
 - ▶ {elementary school, middle school, high school, undergrad, MS, PhD}
- Could use one-hot, but it would erase the order
- Instead, *thermometer code*:
 - ▶ use k features for k values, but turn on the first j of them to encode the j th value
 - ▶ e.g., high school $\rightarrow (1,1,1,0,0,0)$
 - ▶ analogy to the mercury rising in an old-fashioned thermometer
- Can also omit a feature for the first value (assign all zeros) and have $k - 1$ features

Linearly dependent features

- Sometimes a feature will be accidentally duplicated (e.g., two sources provided it w/ different names, or we added a constant feature when one was there already)
- More subtly, a feature might be the sum or difference of two others — e.g., constant vs. one-hot w/o default
- This is no problem w/ regularization
 - ▶ L_2 : if k copies of a feature, we'll assign $\frac{1}{k}$ weight to each
 - ▶ L_1 : any way to split weight is optimal, often prefer sparse
 - ▶ other: will usually do something sane
- But without regularization, optimum might be undefined!
- Diagnose by testing rank of feature matrix X
 - ▶ fix by finding features to drop, or by PCA (later)

Feature range

- Features can have very different typical range of values
 - ▶ e.g., a yes/no question vs. the population of a city
 - ▶ e.g., units: measure distance in mm vs. Mpc
- If no regularization, linear models don't care
 - ▶ multiply feature by 73, regression will divide weight by 73
- But regularization changes this (why?)
- And some optimizers can break (e.g., SGD)
- Two common ways this can happen: location and scale

Location

- Remove location differences by subtracting feature means
 - ▶ feature X_i^t becomes $X_i^t - \hat{\mu}_i$ (called **centering**)
 - ▶ here $\hat{\mu}_i$ is an estimate of the mean (or could use other statistics such as median or trimmed mean)
- Note: must compute $\hat{\mu}_i$ from *training data only*
 - ▶ don't use validation or test data: can't in deployed version, so doing it during development gives wrong answers
 - ▶ $\hat{\mu}_i$ becomes part of our model
- Alternate approach: omit regularization on constant coefficient b in $w \cdot X^t + b$
 - ▶ then b is free to compensate for differences in mean

Scale

- Remove scale differences by dividing by feature scale
 - ▶ feature X_i^t becomes $\frac{1}{\hat{\sigma}_i} X_i^t$ where $\hat{\sigma}_i$ is estimate of scale
 - ▶ e.g., the standard deviation (square root of variance)
 - ▶ or the max absolute value, interquartile range, ...
- As above, must compute $\hat{\sigma}_i$ on *training data only*, and $\hat{\sigma}_i$ becomes part of our model
- If we're transforming location and scale, do location first
 - ▶ scale is more meaningful on centered data

Outliers

- Sometimes a single example will completely dominate the size of a feature
 - ▶ e.g., weight: most examples are mice, one is elephant
 - ▶ or typo: forgot separator, so “030, 001” → “030001”
- Possible fixes:
 - ▶ filter out (delete) the outlying examples
 - ▶ clip the feature (project onto an estimate of the “reasonable” range, e.g., elephant → heaviest mouse)
 - ▶ CDF transform (rank transform): replace feature value by its rank in the training set, then scale to $[0,1]$
 - ▶ whichever we use, has to depend only on training set, becomes part of our model

And more...

- Encode examples w/ a foundation model
- Text → word part embeddings
- Random projection
- Vector quantization
- Positional encoding
- ...

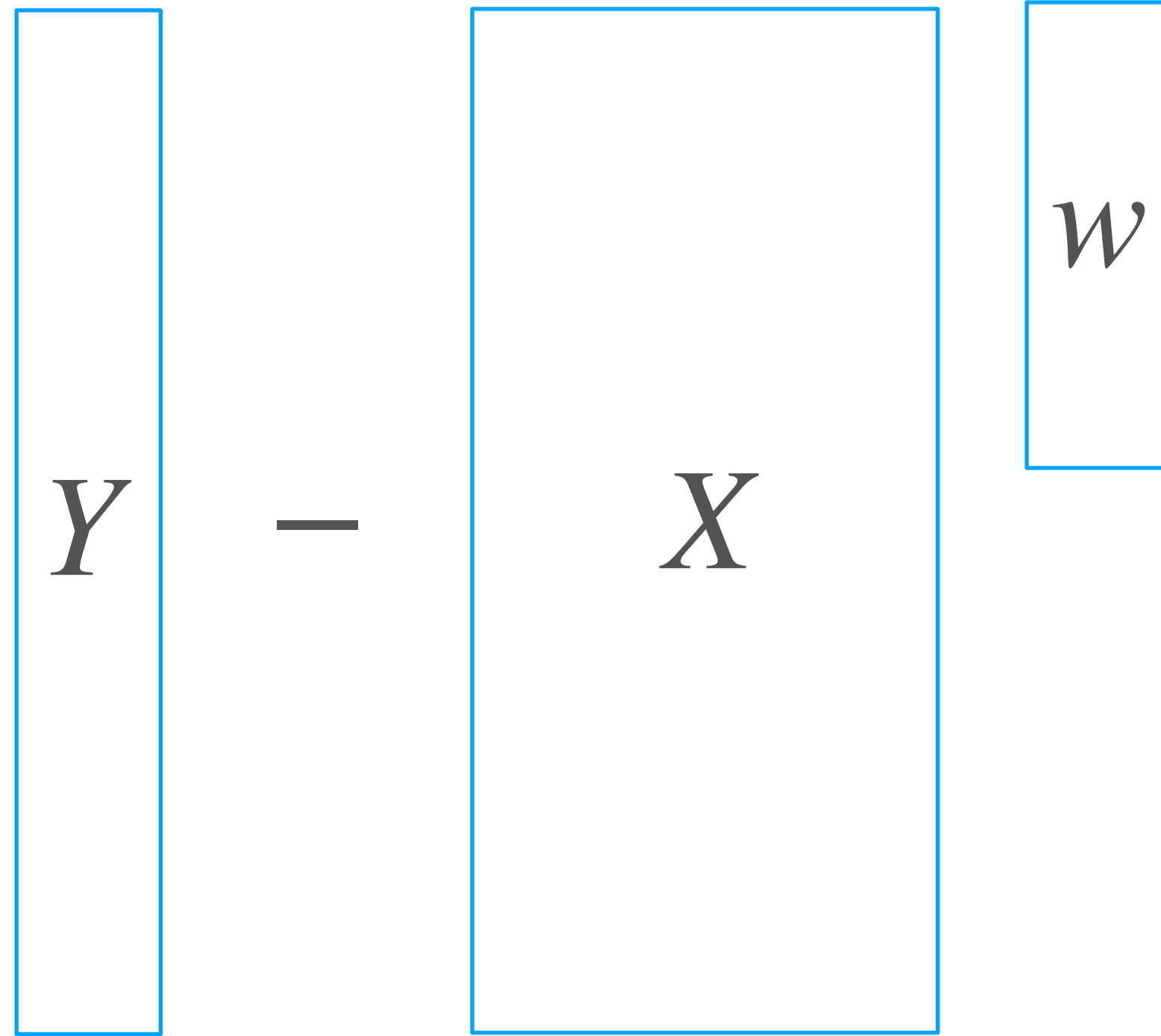
To process or not to process?

- Tempting to apply every possible preprocessing trick just to be sure
- Unfortunately it's not this simple
 - ▶ can lead to overfitting (parameters: location, scale, CDF)
 - ▶ can erase useful information: maybe the feature that didn't vary much is actually not very informative
 - ▶ interpretability
- Probably best to be conservative:
 - ▶ look for problems that influence the fit: outliers, significant variation in scale, ...
 - ▶ fix at the source if possible
 - ▶ pre-process when necessary

Solving it

- To find best w , several options:
 - ▶ minimize symbolically by setting $\nabla_w L$ to 0
 - ▶ sub-choices: QR, SVD, Cholesky
 - ▶ iterative optimizer
 - ▶ sub-choices: stochastic vs. full gradient
 - ▶ 1st-order like SGD or Adam
 - ▶ 2nd-order like Newton
 - ▶ in-between like L-BFGS
 - ▶ tools like momentum, rescaling
 - ▶ choice of covariance form or kernel form

Gradients



- For any of the above, need gradients

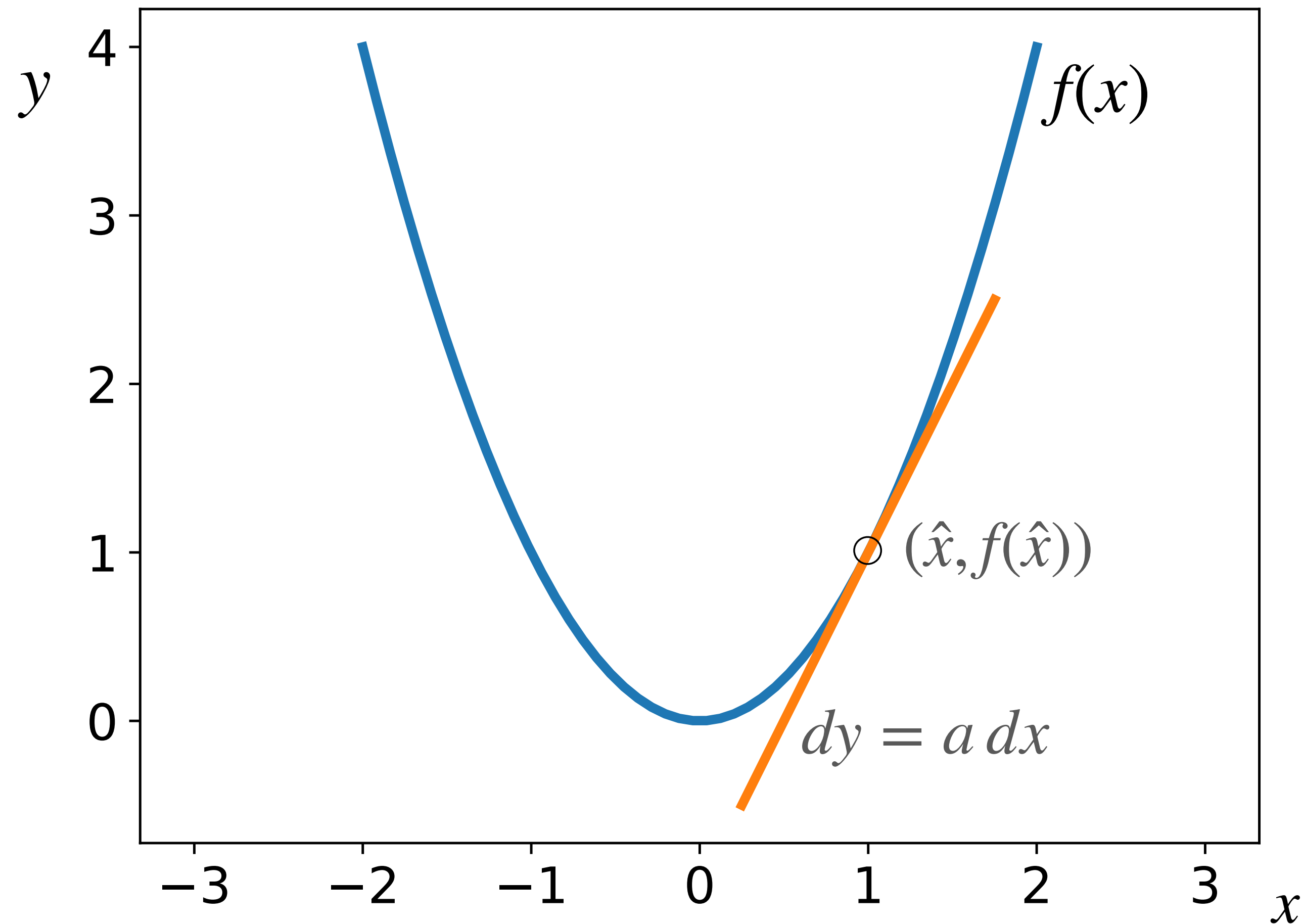
- ▶ $L(w) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - w \cdot X^{(t)})^2$
 $= \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \|Y - Xw\|^2$
 - ▶ $\nabla L(w) =$

Derivatives everywhere

- The way we derive many ML methods:
 - ▶ start from objective to minimize (here, sum of squared errors plus optional regularizer)
 - ▶ take a few derivatives ($0 = d \text{ objective} / dw$)
 - ▶ find optimum (e.g., by setting to 0 and solving, or SGD)
 - ▶ done!
- Will continue to be a common pattern
- Review of intuition and tools for high-d derivatives, so we can do it fast

Derivatives notation

dx, dy are called
differentials



$$m = n = 1$$

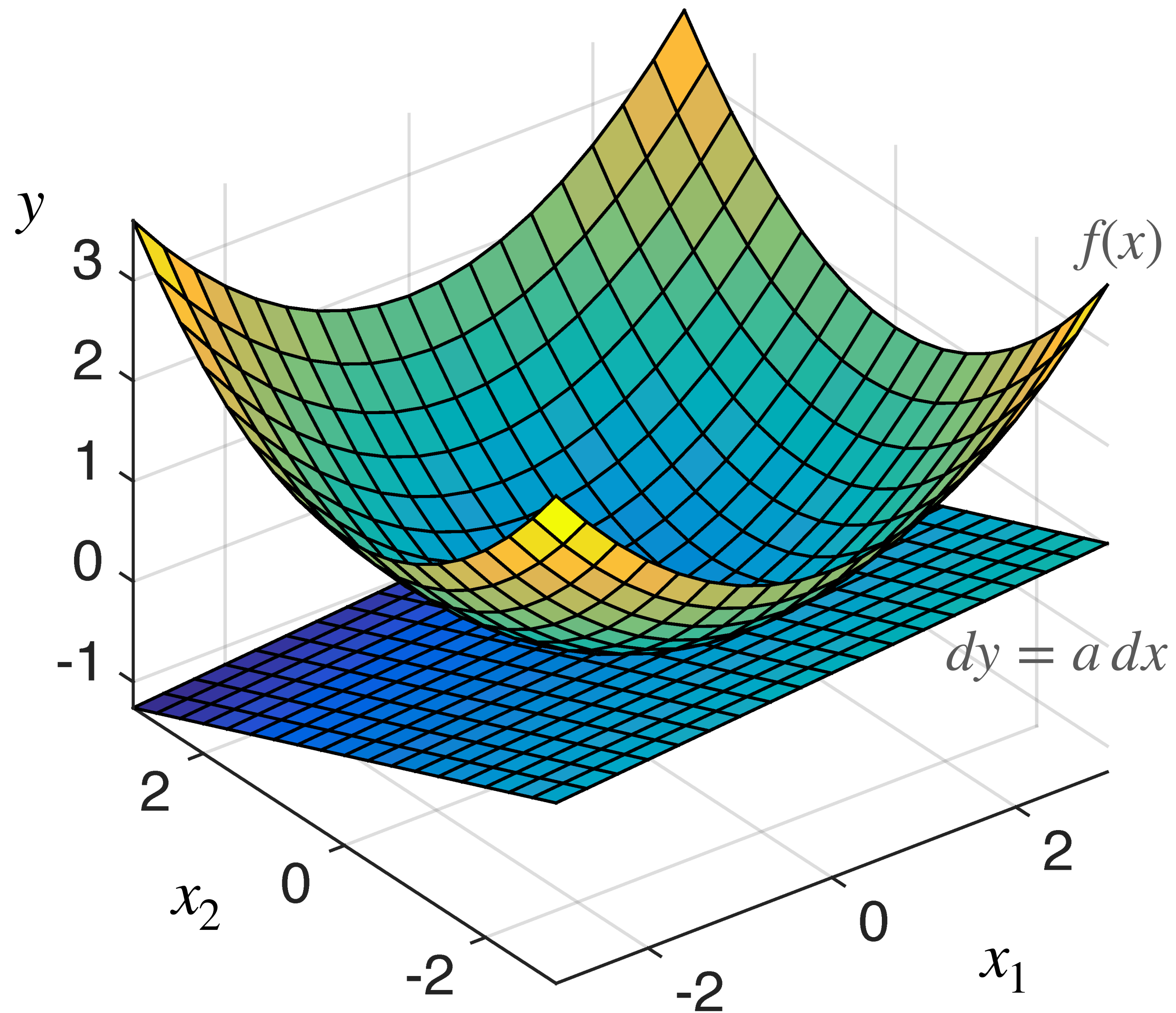
$$dy = y - f(\hat{x})$$
$$dx = x - \hat{x}$$

- Function $y = f(x)$
 - ▶ $x \in \mathbb{R}^n, y \in \mathbb{R}^m, f \in \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Derivative is a *linear fn* that *locally* approximates f near $\hat{x}$
 - ▶ $dy = a dx$, where $a \in \mathbb{R}$ is the derivative

Compare older vs. newer notation

- Older version:
 - ▶ $\frac{dy}{dx}$ is atomic: derivative of y wrt x
 - ▶ dx and dy are meaningless on their own
 - ▶ focused on infinitesimals: change in x or $y \rightarrow 0$
- Newer version:
 - ▶ dx and dy have separate meaning: terms in a linear function (the Taylor expansion)
 - ▶ dx and dy can be any size: linear function is well defined even for large changes in x or y (though accuracy of Taylor approximation can be bad if dx , dy are big)
- Heuristic to convert: “divide through” or “multiply through” by dx (only makes sense in scalar case, but often succeeds in converting notation anyway)

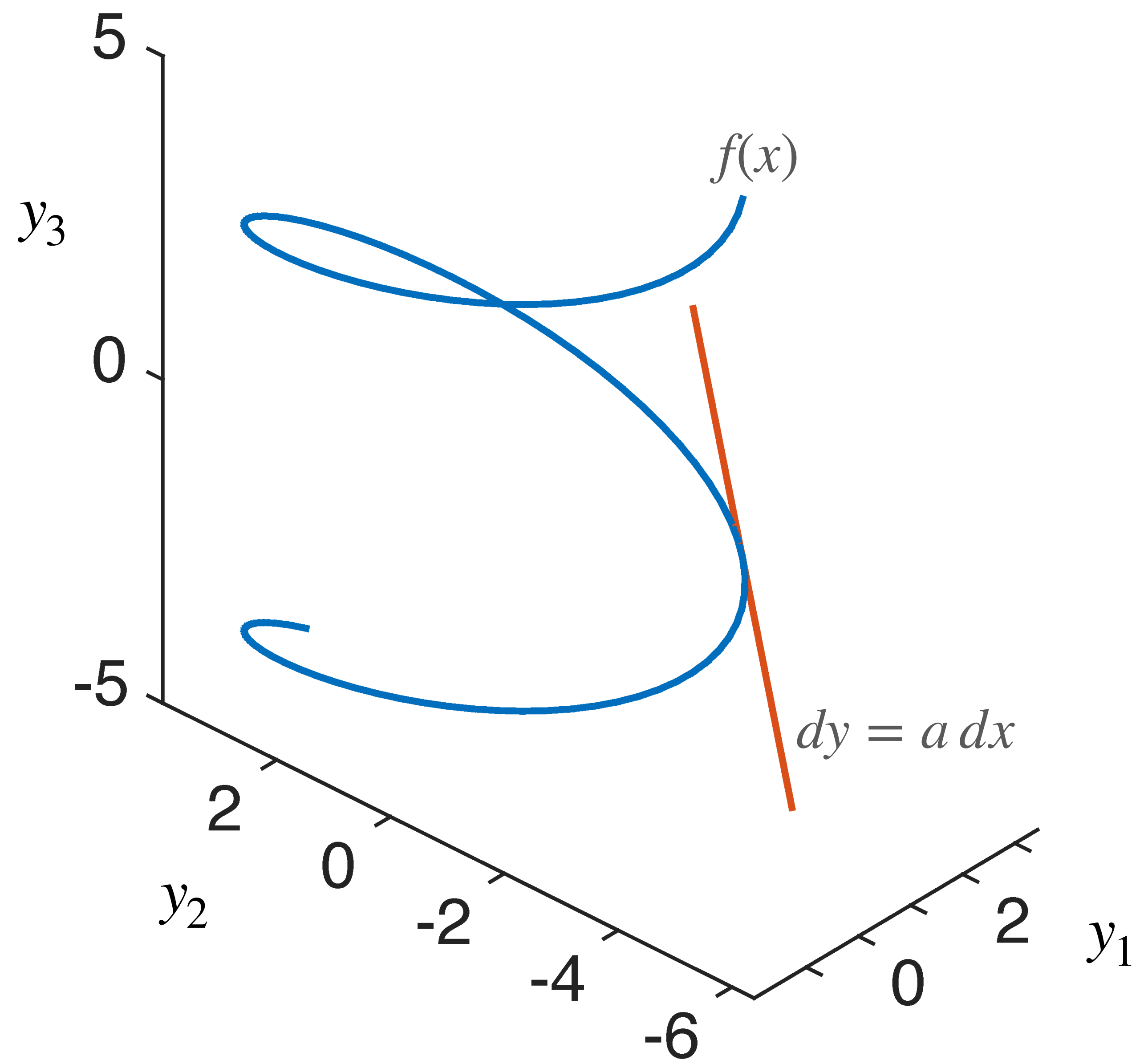
Higher dimensions



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$
$$m = 1, n = 2$$

Function $y = f(x)$ $dy = a dx$ $a \in \mathbb{R}^{1 \times 2}$

*Higher
dimensions*



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$
$$m = 3, n = 1$$

Function $y = f(x)$ $dy = a dx$ $a \in \mathbb{R}^{3 \times 1}$

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$ds = a dt$	$ds = u^T dv$	$ds = \text{tr}(M^T dN)$
Vector	$du = v ds$	$du = M dv$	$\times$
Matrix	$dM = N ds$	$\times$	$\times$

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- $\times$ means no common notation; see torch.einsum

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$d(\text{slope})/dt$	(gradient)	$ds = \text{tr}(M^T dN)$
Vector	(velocity)	(Jacobian)	$\times$
Matrix	$dM = N ds$	$\times$	$\times$

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- $\times$ means no common notation; see torch.einsum

Inner product

- With scalar output s , for any shape of input X , can write $ds = \langle A, dX \rangle$ where A, X, dX all have same shape
 - ▶ $\langle \cdot, \cdot \rangle$ is ***inner product***, in our case $\sum_{\text{index lists } I} A_I dX_I$
 - ▶ e.g., if A, dX are $m \times n$ matrices, then
$$\langle A, dX \rangle = \sum_{i=1}^m \sum_{j=1}^n A_{ij} dX_{ij}$$
- Expressions on previous slide are equivalent to above
 - ▶ most complex is $\text{tr}(M^\top dN)$
 - ▶ tr is sum of diagonal elements of a square matrix
$$[M^\top dN]_{jk} =$$
 - ▶ sum of diagonal elements means set $j = k$, sum over j :

Shape conventions

- Derivative = coefficient of ds, dt, dv, dN on RHS
- Ambiguity: can write $\langle M, dN \rangle$ or $\text{tr}(M^\top dN)$
 - ▶ is the derivative M or $M^\top$?
- Convention: x, y , and derivative $\frac{dy}{dx}$ are **tensors** (multidimensional arrays of numbers)
- In derivative $\frac{dy}{dx}$:
 - ▶ the dimensions of y come first, then x
 - ▶ in same order as dimensions of y and x
- For example: if y is 3×2 and x is $4 \times 6 \times 5$ then $\frac{dy}{dx}$ is $3 \times 2 \times 4 \times 6 \times 5$

Special cases

- If y is a scalar (dimensions $()$), then $\frac{dy}{dx}$ is the same shape as x
 - ▶ derivative of a scalar wrt a 5×3 matrix is a 5×3 matrix
- If x and y are vectors, $\frac{dy}{dx}$ (the Jacobian) is the same shape as a linear function from x to y
 - ▶ derivative of a 3-vector wrt a 6-vector is a 3×6 matrix

Useful identities

- $\text{scalar} = \text{scalar}^\top = \text{tr}(\text{scalar})$

- Trace rotation

- ▶ $\text{tr}(ABC) = \text{tr}(CAB)$ when A, B, C have compatible dimensions

compatible dimensions:

- 2nd dimension of A = 1st dimension of B
- 2nd dimension of B = 1st dimension of C
- 2nd dimension of C = 1st dimension of A

- Transpose-Hadamard

- ▶ $A \circ B = \text{Hadamard (componentwise) product,}$

$$[A \circ B]_{ij} = A_{ij}B_{ij}$$

- ▶ $A^\top(B \circ C) = (A \circ B)^\top C$

Tools for derivatives: linearity, chain, product

- Linearity:
 - ▶ $d(ax + b) = d(ax) + d(b) = a \, dx, \quad d(AX + B) = A \, dX, \dots$
- Chain rule for $f(g(x))$
 - ▶ old notation: $\frac{df}{dx} = \frac{df}{dg} \frac{dg}{dx}$ (“cancel the dg ”)
 - ▶ new: if $df = A \, dg, \, dg = B \, dx$ then $df =$
- Product rule for $f(x) \, g(x)$
 - ▶ $d(fg) = df \, g + f \, dg$
 - ▶ works for any kind of product (matrix product, vector cross product, elementwise (Hadamard) product, vector convolution, Khatri-Rao product ...)
 - ▶ unlike scalar case, order can matter (products are not necessarily commutative)

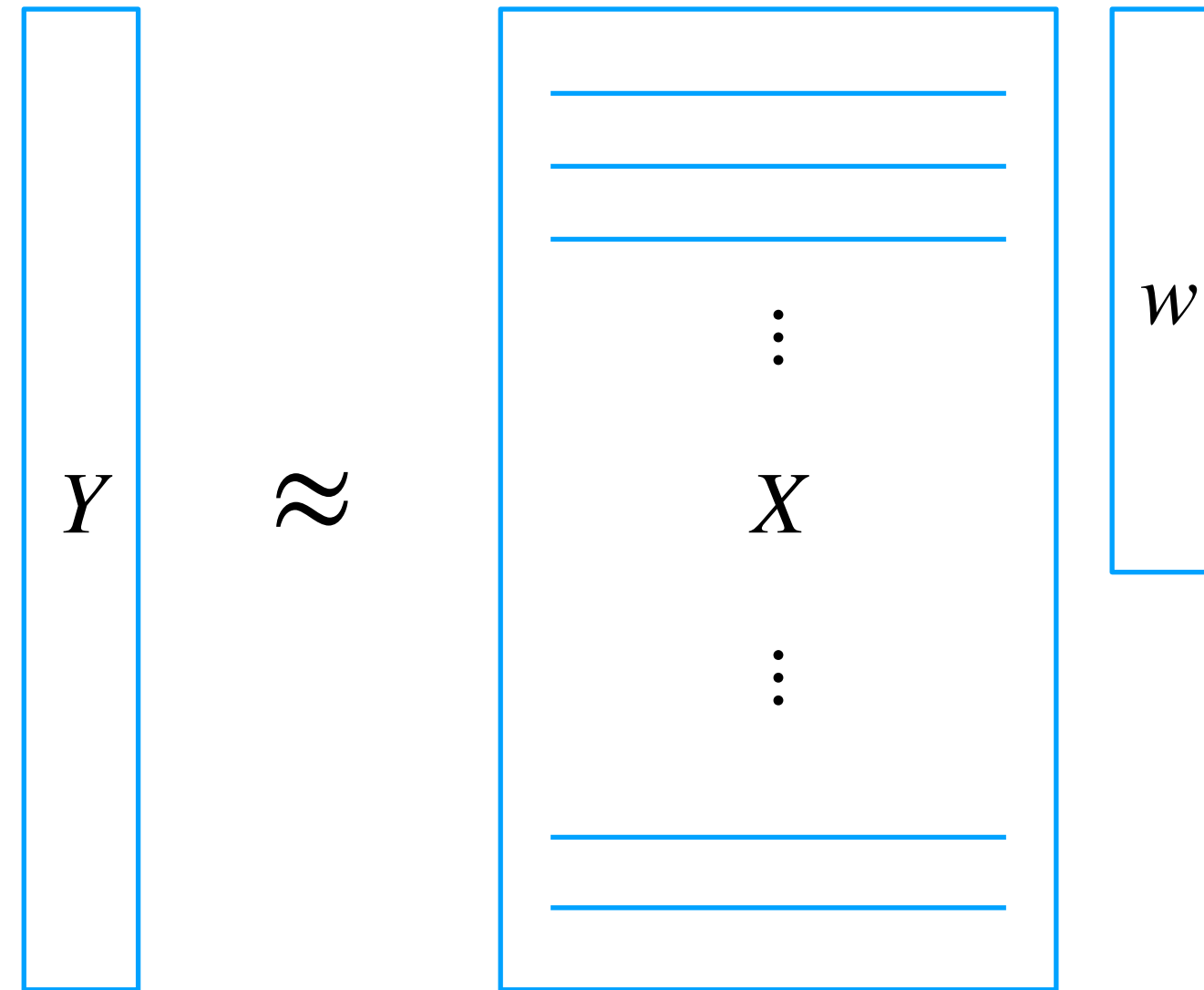
***Gradient
practice:
Linear
regression
in one slide***

$$L = \|Y - Xw\|^2 = (Y - Xw)^\top (Y - Xw)$$

$$L = Y^\top Y - 2Y^\top Xw + w^\top X^\top Xw$$

Exact solution for regression and ridge regression

$$\begin{aligned} X &\in \mathbb{R}^{T \times d} \\ Y &\in \mathbb{R}^T \\ w &\in \mathbb{R}^d \end{aligned}$$



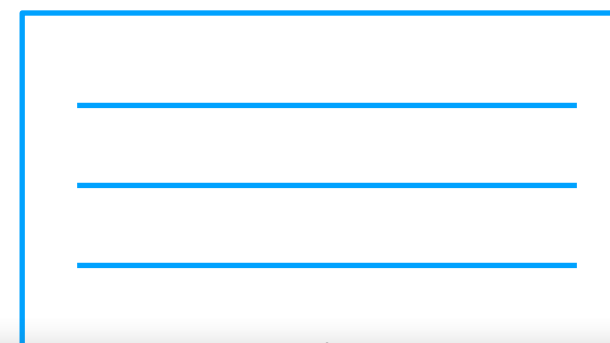
- Predictions are $\hat{Y} = Xw$
- Objective is $\frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \|Y - Xw\|^2$
- Gradient is $\lambda w + X^\top (Xw - Y)$
- Exact solution is $(\lambda I + X^\top X)w = X^\top Y$
 - ▶ implemented in `torch.linalg.lstsq`

Exact solution for regression and ridge regression

$$X \in \mathbb{R}^{T \times d}$$

$$Y \in \mathbb{R}^T$$

$$w \in \mathbb{R}^d$$



- Predict

- Objective

- Gradient

- Exact solution

Why use `torch.linalg.lstsq`? Handles (many) special cases: over- or under-determined, rank-deficient, poor conditioning

It would be several days of lectures to describe algorithms and best practices for all of these

Please remember: don't write `inv()` unless you are sure none of the special cases will bite you

- ▶ implemented in `torch.linalg.lstsq`